

LOSSLine

Final Product Requirements Document

AI Operational Intelligence for Multi-Location Restaurants

Team: Rakshith A, Raghuttama K G, Chandranshu Kumar, Gaurav Kumar

Document purpose: Consolidated final PRD incorporating the original requirements plus the finalized event-ingestion, streaming, detection, agent-loop, memory, action, verification, and live-demo architecture.

Build principle: LOSSLine is not a dashboard with an LLM attached. It is an event-driven, evidence-grounded, closed-loop operational intelligence system.

1. Executive Summary

LOSSLine monitors multi-location restaurant operations by continuously ingesting fragmented operational events from POS, inventory, staffing, reviews, delivery, payments, kitchen systems, and external signals such as weather. It converts heterogeneous source events into a common event model, streams them through a detection layer, and triggers an agentic investigation only when meaningful anomalies or incidents are detected.

The system then correlates evidence, identifies likely root cause, scores confidence, estimates operational/revenue impact, recommends a concrete action, routes consequential actions through human approval, and checks later whether the action actually improved the targeted metric. Outcomes are stored so future investigations and the AI Copilot can use previous incidents and results.

This preserves the strongest idea from the original PRD: LOSSLine explains why an issue is happening rather than merely flagging that it happened, and closes the loop by checking whether its own recommendation worked. The source PRD explicitly frames this as the key distinction from a normal dashboard. ■filecite■turn0file0■L5-L11■

2. Problem Statement

Multi-location restaurant managers operate across siloed systems. A sales dip, stock variance, delivery delay, payment failure, or negative-review cluster rarely has a single obvious cause. Existing dashboards visualize symptoms, but managers still perform the cross-system correlation manually.

- Revenue loss caused by delayed detection and response.
- Manual correlation across POS, inventory, staffing, reviews, delivery, payments, and external conditions.
- Reactive rather than proactive operations.
- Stockouts, excess refunds, food waste, inconsistent service, and technology failures.

The original PRD identifies the same gap: traditional BI shows what happened but does not reliably correlate sources, explain root cause, or recommend and validate an action. ■filecite■turn0file0■L12-L26■

3. Product Vision

NORMAL DASHBOARD

Data → Charts → Manager investigates → Manager acts

LOSSLine

Data → Stream → Detect → Investigate → Explain → Estimate Loss
→ Recommend → Human Approval → Execute → Verify → Learn

The core product question is not “**What changed?**” but “**What is going wrong, why is it happening, how much is it costing, what should we do, and did the intervention work?**”

4. Scope

Goals

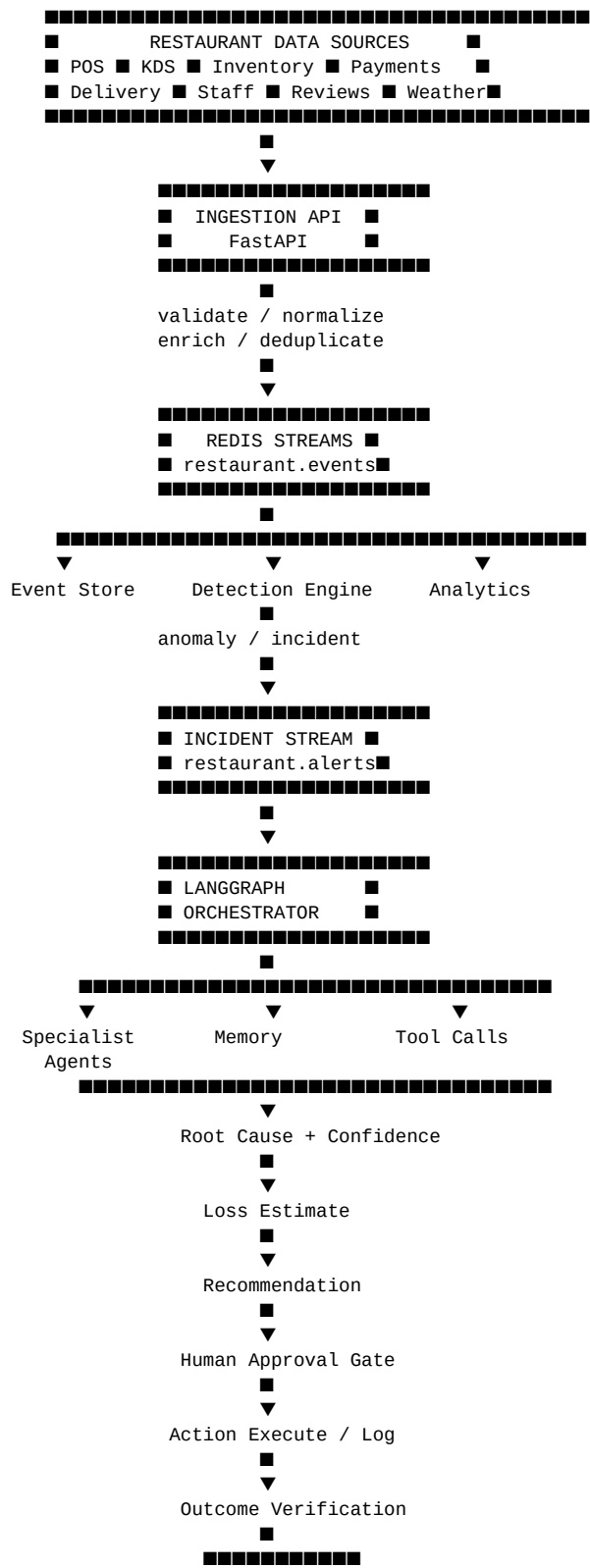
- Detect operational anomalies across at least four data sources.
- Explain root cause using correlated evidence rather than free-floating LLM text.
- Attach a confidence score to every incident.
- Generate a concrete recommendation with expected impact.
- Track whether the recommendation worked.
- Keep humans in the loop for consequential actions.
- Make the entire event-to-outcome pipeline visible during the demo.

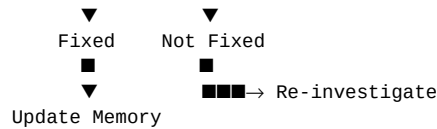
Non-Goals for the Hackathon

- Fully autonomous execution of medium/high-risk actions without approval.
- Real POS/inventory integrations; synthetic event generation is the primary demo source.
- Production-grade multi-tenant security hardening.

These non-goals are consistent with the original PRD. ■filecite■turn0file0■L33-L46■

5. Core System Architecture





6. Data Ingestion & Event Streaming

6.1 Why Streaming

The restaurant does not upload one combined CSV. Each source produces individual events as things happen. The original PRD explicitly requires an event-driven rather than batch model: an order, stock update, review, or weather change arrives individually, and the affected specialist logic can re-evaluate when new evidence arrives.

filecite turn0file0 L168-L177

For the hackathon, Redis Streams are the recommended event-streaming layer because they provide a simple, visible, lightweight mechanism for the demo while allowing multiple consumers to process the same events.

6.2 Raw Source Events

```

POS
{
  "orderId": "ORD-9211",
  "status": "cancelled",
  "store": 17,
  "amount": 749
}

PAYMENT
{
  "transaction_id": "TX-8821",
  "status": "failed",
  "method": "upi",
  "amount": 749
}

INVENTORY
{
  "sku": "CHICKEN_01",
  "previous_qty": 42,
  "new_qty": 29,
  "unit": "kg"
}

WEATHER
{
  "rainfall_mm": 18.4,
  "temperature": 24,
  "condition": "heavy_rain"
}
  
```

6.3 Canonical LOSSLine Event

```

{
  "event_id": "evt_1005",
  "restaurant_id": "store_17",
  "timestamp": "2026-08-08T12:43:17Z",
  "source": "payment",
  "event_type": "payment.failed",

  "entity": {
    "type": "restaurant",
    "id": "store_17"
  },

  "data": {
    "transaction_id": "TX-8821",
    "amount": 749,
    "method": "upi"
  },

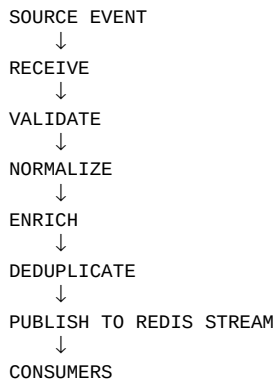
  "metadata": {
    "schema_version": "1.0",
  }
}
  
```

```

    "ingested_at": "2026-08-08T12:43:18Z"
  }
}

```

6.4 Ingestion Pipeline



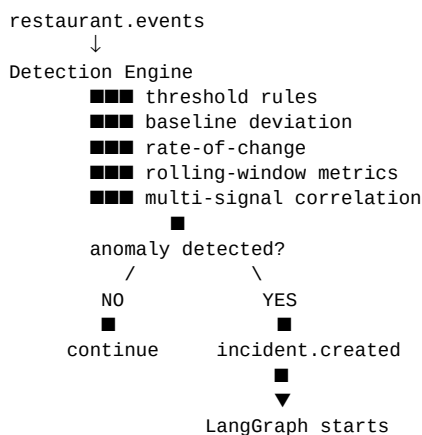
- Validate required IDs, timestamps, source type, schema, and numeric fields.
- Normalize different source naming conventions into one internal event schema.
- Enrich with restaurant metadata, current rolling metrics, and relevant context where available.
- Deduplicate using event_id/source identifiers so repeated delivery does not create false incidents.
- Publish to a Redis Stream for asynchronous consumers.

6.5 Stream Layout

Stream	Purpose
restaurant.events	Normalized operational events from all sources.
restaurant.alerts	Anomalies and incident-created events.
restaurant.actions	Recommendations, approvals, executions, and action logs.
restaurant.verification	Outcome checks, recovery signals, and resolution events.

7. Detection Engine — The Agent Trigger

The detection layer is deliberately separated from the LLM agent. It should usually be deterministic/statistical code, not an LLM call. Its job is not to explain the incident. Its job is to decide whether the incoming stream contains enough evidence to create an incident.



Example

Normal payment failure rate at Store #17: 2-4%

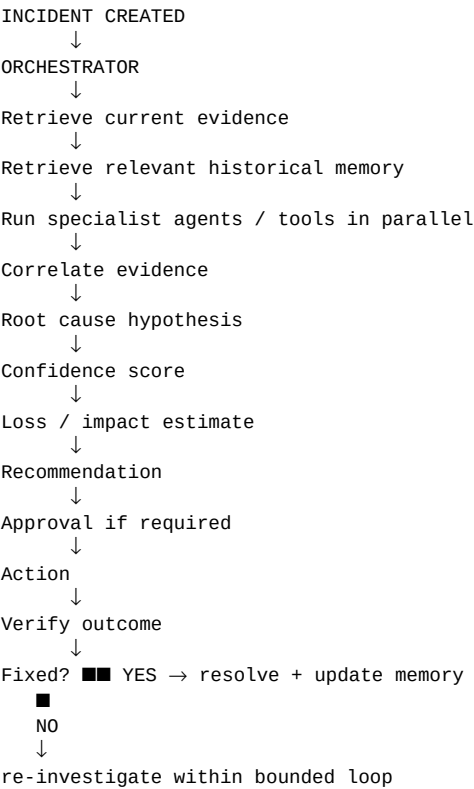
```
Observed rolling window:
120 payments
21 failures
17.5% failure rate

Detector:
17.5% > baseline × configured threshold

Result:
incident.created
severity = critical
trigger = payment_failure_rate
expected = 0.028
actual = 0.175
```

This prevents an LLM call for every raw event. Thousands of routine events can be processed cheaply, while only meaningful incidents trigger the expensive agentic investigation.

8. Agentic Investigation Loop



Specialist Agents

Agent	Responsibility
POS Agent	Orders, cancellations, sales, order volume, POS health.
Inventory Agent	Stock movement, variance, waste, stockouts.
Reviews Agent	Complaint/review clusters, rating changes, customer experience.
Weather Agent	Rain, temperature, external conditions affecting demand/delivery.
Delivery Agent	Delay, cancellation, driver/platform performance.
Payment Agent	Failure rate, gateway latency, payment-method anomalies.
Staff Agent	Attendance, staffing level, labor efficiency and shift anomalies.

9. Agent State vs Memory

Short-Term State

LangGraph shared state represents the current investigation: incident ID, restaurant, observations, hypotheses, evidence, confidence, retry count, recommended action, and status. It exists to coordinate the current graph execution.

```
{
  "incident_id": "INC-1029",
  "restaurant_id": "store_17",
  "anomaly": "payment_failure_spike",
  "observations": [],
  "hypotheses": [],
  "evidence": [],
  "confidence": 0.0,
  "retry_count": 0,
  "status": "investigating"
}
```

Long-Term Memory

For the hackathon, use structured retrieval over incidents and outcomes rather than introducing a vector database unnecessarily. The original PRD explicitly recommends retrieval from incidents + outcomes tables and states that branch/root-cause/signal-combination keys are specific enough for SQL matching. [filecite turn0file0 L432-L464](#)

Memory type	What it stores	Recommended implementation
Episodic	What happened, what action was taken, and what happened afterwards	Postgres incidents + outcomes
Semantic/operational	Known store characteristics, signal relationships, recurring patterns	Postgres structured knowledge
Procedural	Investigation playbooks, rules, action policies.	Code/config + prompts
Session state	Current investigation context.	LangGraph state

Memory Retrieval Example

```
New incident:
Store #17 + inventory shortage + stock_variance spike

Query:
SELECT past incidents/outcomes
WHERE branch = 'store_17'
      AND root_cause_category = 'inventory'
ORDER BY created_at DESC
LIMIT 5

Memory:
Previous action = Emergency Stock Transfer
Outcome = Worsened

Agent:
Do not blindly repeat the previous action.
Consider an alternative and cite the previous outcome.
```

10. Confidence & Uncertainty

Confidence	Behavior
≥ 0.75	High confidence: full explanation + recommendation; eligible for human-approval/auto-execute flow depending on risk.
0.50–0.75	Medium confidence: explanation + recommendation, but always requires human approval.
< 0.50	Low confidence: abstain from recommendation; show anomaly detected with insufficient corroborating evidence.

Low-confidence abstention is a product feature. The original PRD explicitly treats “we don't know yet” as the correct behavior when evidence is insufficient. [filecite turn0file0 L130-L143](#)

11. Bounded Agent Loops

The correlation agent can request a wider evidence window when confidence is low, but it must not loop indefinitely.

```
Attempt 1:
Initial correlation pass.

If confidence < 0.50:
Attempt 2:
Widen evidence window by ±1 hour and re-run correlation.

If confidence still < 0.50:
STOP.
Emit low-confidence / monitor-only incident.
No recommendation.
```

Maximum retry loops per incident: 2. A retry counter in LangGraph state and a hard node-visit ceiling provide bounded execution. This matches the original PRD's loop-limit design. ■filecite■turn0file0■L99-L110■

12. Action, Human Approval, Verify, and Fixed

12.1 Action

The recommendation agent converts the investigation into a concrete action and expected impact. Example: "Review score is falling because delivery delays increased; pause new delivery orders for 30 minutes and notify affected customers."

12.2 Human Approval

Low-risk informational actions can be passive. Medium-risk actions require manager approval. High-risk actions require manager approval plus visible urgency/expiry handling. The original PRD specifies a LangGraph human-in-the-loop interrupt and an approval queue. ■filecite■turn0file0■L144-L166■

12.3 Execution

For the hackathon, real operational systems are not modified. The action is logged or executed through a simulator. This makes the closed loop visible without pretending to have production control of restaurant infrastructure.

12.4 Verify

Every approved recommendation stores an expected_impact and check_after timestamp. A scheduled outcome-tracking worker later re-queries the targeted metric and compares it against the pre-action baseline. The result is Improved, No change, or Worsened. ■filecite■turn0file0■L208-L227■

```
BEFORE
Payment failure rate = 17.4%

ACTION
Restart payment connector

AFTER
Payment failure rate = 2.1%
Gateway latency = 1.2s

VERIFY
Expected direction achieved
→ Outcome = Improved
→ Incident = Resolved
```

12.5 If the Fix Did Not Work

```
Action executed
↓
Verify
↓
Still abnormal
↓
NOT FIXED
↓
Create new evidence set
↓
```


Re-investigate
↓
Alternative hypothesis/action

13. Notification Layer

Notifications are driven by urgency. Low urgency stays in the in-app feed. Medium urgency creates a push notification and approval-queue entry. High urgency can use push plus SMS/email and optionally Slack/Teams. Duplicate alerts should update an existing incident rather than spam the manager. The live dashboard uses WebSocket updates.

■filecite■turn0file0■L178-L201■

14. Live Demo Architecture

The demo must use the real ingestion path rather than a fake frontend animation. A scenario runner replays timestamped JSON events by calling POST /events at realistic intervals. WebSocket messages expose pipeline transitions to the dashboard. This is explicitly defined in the original PRD's Demo Mode section.

■filecite■turn0file0■L394-L431■

```
[Run Demo Scenario]
↓
Scenario Runner
↓
POST /events
↓
FastAPI ingestion
↓
Redis Stream
↓
Detection
↓
incident.created
↓
LangGraph
↓
Specialist agents light up
↓
Evidence appears
↓
Correlation fires
↓
Confidence updates
↓
Recommendation appears
↓
Approval
↓
Simulated action
↓
New events
↓
Verification
↓
Improved / No change / Worsened
```

Demo Scenario Example — Weather-Driven Delivery Failure

1. Normal order traffic
2. Heavy rain events arrive
3. Delivery latency increases
4. Customer complaints increase
5. Cancellations increase
6. Detection creates incident
7. Weather + Delivery + Reviews agents collect evidence
8. Correlation identifies weather-driven delivery delay
9. Confidence = 0.91
10. Recommendation:
 - pause new delivery orders for 30 minutes
 - + notify affected customers
11. Manager approves
12. Simulator executes action
13. New events show cancellations falling
14. Outcome = Improved

- 15. Incident resolved
- 16. Incident/outcome saved for future memory

15. API Surface

Endpoint	Purpose
POST /events	Primary ingestion endpoint for every normalized event.
POST /demo/run?scenario=...	Replay a seeded live scenario through the real ingestion path.
POST /demo/reset	Clear demo incidents/outcomes for a clean replay.
GET /incidents	Incident feed.
GET /incidents/{id}	Evidence, confidence, recommendation, action, and outcome.
GET /analytics/summary?range=...	Pre-aggregated analytics for dashboard charts.
POST /incidents/{id}/approve	Manager approves an action.
POST /incidents/{id}/reject	Manager rejects an action.
WebSocket /ws	Real-time event/agent/pipeline UI updates.

16. Data Model

Table	Key fields
events	event_id, restaurant_id, source, event_type, timestamp, payload JSONB
incidents	incident_id, restaurant_id, trigger, severity, confidence, root_cause, status
evidence	incident_id, event_id, source, observation, relevance
recommendations	incident_id, action, expected_impact, urgency, approval_status
actions	recommendation_id, approved_by, executed_at, execution_status
outcomes	incident_id, check_after, baseline, post_metric, tag
restaurants	restaurant_id, name, region, metadata
scenario_events	scenario_id, sequence, timestamp_offset, event_json

17. Recommended Tech Stack

Layer	Technology	Why
Agent orchestration	LangGraph	Parallel fan-out, shared typed state, conditional edges, human-in-the-loop.
LLM	Gemini	Grounded reasoning and function calling for structured recommendations.
Backend / ingestion	FastAPI	Async, typed validation, clean event API.
Event streaming	Redis Streams	Lightweight real-time event bus suitable for the demo.
Database	PostgreSQL	JSONB for event payloads and relational incident/outcome joins.
Synthetic data	Python + Faker	Seeded, reproducible restaurant scenarios.
Frontend	React + existing dashboard	Incident feed, approval queue, live pipeline.
Realtime UI	WebSocket	Live event/agent status without polling.
Scheduling	APScheduler / cron-style worker	Outcome checks and day-close jobs.

Layer	Technology	Why
Tracing	LangSmith	Agent graph tracing and judge-facing debugging.
Charts	Recharts	Analytics visualization.
Deployment	Docker	Reproducible demo environment.

The original PRD recommends LangGraph, Gemini, FastAPI, Postgres, Python/Faker, React, APScheduler/cron-style scheduling, and LangSmith; the streaming layer and explicit event schema above formalize the event-driven design discussed during the architecture refinement. ■filecite■turn0file0■L48-L93■

18. Frontend / Dashboard

Live Operations View

- Active incidents ranked by urgency and confidence.
- Live event stream.
- Live pipeline showing ingestion → detection → agents → recommendation → verification.
- Branch/location health cards.
- Estimated revenue at risk.

Incident Detail

- Root cause.
- Evidence trail with source events.
- Confidence bar and tier.
- Loss/impact estimate.
- Recommendation and expected impact.
- Approval controls.
- Action status and outcome.

Analytics

- Incidents over time.
- Incidents by location.
- Root-cause breakdown.
- Confidence distribution.
- Recommendation outcome rate.
- Detection-to-recommendation latency.
- Approval funnel.

The original PRD specifies these analytics widgets and recommends a backend aggregation endpoint with Recharts on the frontend. ■filecite■turn0file0■L343-L393■

19. AI Copilot with Memory

The Copilot is context-aware rather than a generic chatbot. It receives the current incident/page context and retrieves similar historical incidents and outcomes before answering. The original PRD specifically recommends retrieval rather than fine-tuning and suggests surfacing a small “based on N past incidents” indicator for transparency.

filecite turn0file0L432-L468

Manager:
"Why is Store #17 losing money?"

Copilot context:
Current incident + current evidence

Memory retrieval:
Past Store #17 incidents
Same root-cause category
Same signal combination

Answer:
Current evidence + relevant past outcomes
+ uncertainty / confidence

20. Global Incident Timeline

A chronological cross-location incident feed answers “what has been happening across the chain?” It should support filtering by branch, confidence tier, root-cause category, and date range, and deep-link into an incident's detailed evidence view. This is distinct from analytics: analytics shows trends; the timeline shows individual events over time.

filecite turn0file0L473-L496

21. Non-Functional Requirements

Requirement	Target
Detection-to-recommendation latency	< ~30 seconds for demo dataset
Explainability	Every recommendation traceable to source events/evidence.
Bounded execution	Retry cap + hard node-visit ceiling.
Auditability	Approval, rejection, execution, and outcome timestamps logged.
Realtime UI	WebSocket updates for live incident/demo pipeline.
Demo reproducibility	Seeded scenarios with reset and speed control.

These targets are based on the original PRD's non-functional requirements. filecite turn0file0L250-L258

22. Phased Build Plan

Phase	Deliverables
1 — Foundation	Event schema, POST /events, Postgres tables, synthetic generator, ingestion tests.
2 — Specialist agents	POS, Inventory, Reviews, Weather, Payment/Delivery agents; parallel LangGraph nodes.
3 — Detection + correlation	Rolling metrics, anomaly rules, correlation, confidence, retry limit, abstention.
4 — Recommendation + approval	Recommendation logic, risk tiers, approval queue, human-in-loop interrupt.
5 — Outcome tracking	expected_impact, check_after, scheduled verification, Improved/No change/Worsened.
6 — Demo polish	Live event simulator, WebSocket pipeline, incident feed, analytics, timeline, reset/speed control.
7 — Stretch	Confidence recalibration from historical outcomes; richer Copilot memory.

This sequencing follows the original PRD's phased roadmap, with the streaming/event-trigger mechanism and live demo made explicit. filecite turn0file0L266-L303

23. Success Metrics

- % of scripted anomalies correctly detected and correlated.
- Separation between confidence scores for true-positive and false-positive scenarios.
- Time from anomaly injection to recommendation.
- % of high-confidence recommendations later confirmed as Improved.
- Percentage of incidents with complete evidence trails.
- Percentage of medium/high-risk actions correctly routed through approval.

The first four measures are directly aligned with the original PRD's evaluation metrics.

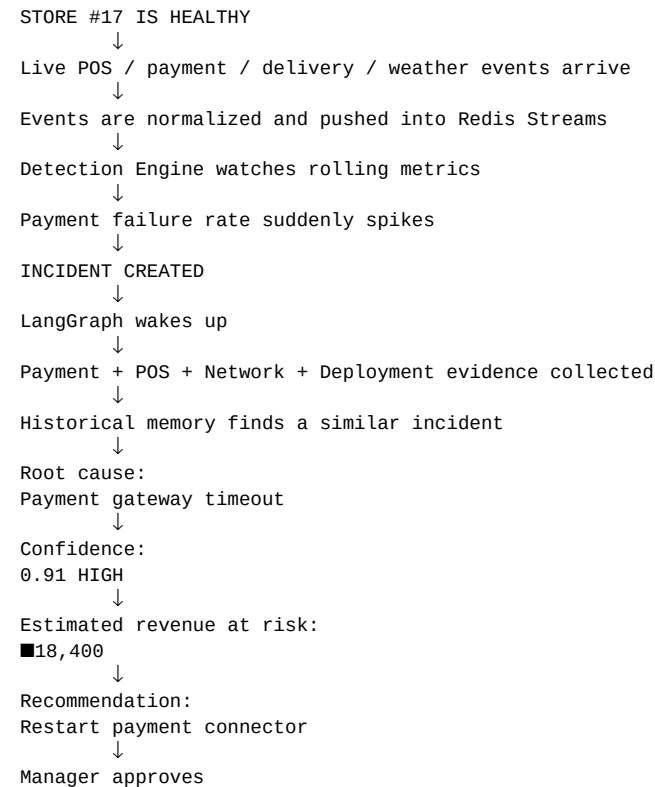
filecite turn0L259-L265

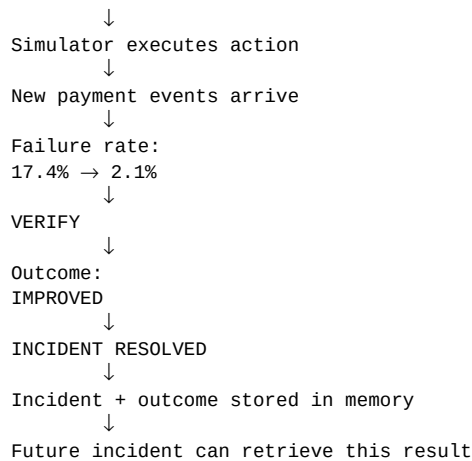
24. Key Risks & Mitigations

Risk	Mitigation
LLM gives plausible but unsupported root cause	Ground the LLM strictly in EvidenceState; no facts outside retrieved evidence.
Agent loops forever	Retry cap + max graph node visits.
Too many false alerts	Baseline/rule thresholds, deduplication, incident aggregation, confidence tiers.
Demo does not visibly show agent work	Live event simulator + WebSocket pipeline visualization.
Action is unsafe	Risk-tiered human approval; no silent medium/high-risk execution.
Recommendation sounds good but fails	Outcome tracking and Improved/No change/Worsened result.
Notification fatigue	Deduplicate and update existing incident alerts.
Insufficient evidence	Explicit abstention below confidence threshold.

These mitigations preserve the original PRD's emphasis on evidence grounding, bounded loops, uncertainty handling, human approval, and outcome tracking. filecite turn0L305-L320

25. Final Demo Story





26. The One-Line Product Definition

LOSSLine is an event-driven, multi-agent operational intelligence platform that detects restaurant anomalies, correlates live and historical evidence to find root cause, estimates business loss, recommends and safely routes remediation, and verifies whether the intervention actually worked.

27. What Makes It Different

Normal Dashboard	LOSSLine
Shows metrics	Interprets events and incidents
Shows symptoms	Correlates evidence to root cause
Manager investigates manually	Agents investigate across sources
LLM may summarize data	LLM is constrained by structured evidence/state
Recommendation ends the flow	Recommendation starts an outcome check
No memory of outcomes	Past incidents/outcomes influence future reasoning
Alerts can be noisy	Deduplicated, confidence-tiered incidents
Static architecture	Live event-driven pipeline visible in demo

This final version retains the original PRD's central design — multi-agent correlation, evidence-grounded confidence, risk-tiered human approval, and closed-loop outcome tracking — while making the event ingestion, Redis streaming, detection trigger, agent state/memory, action/verification cycle, and live demo path explicit.

filecite turn0file0L27-L32